Aahil Ali (aaa387)
Cion Kim (ck758)
Montserrat McCoy (mm3348)

Final Report

Robot Design and Strategy Overview

Our robot was designed to gather as many cubes as possible within the one-minute competition time. Since cubes count as long as they are completely within the robot's perimeter, we focused on making a simple structure that could guide cubes into the robot and keep them there.

Mechanically, the robot used cardboard walls around the back and sides of the frame. These walls acted as a barrier to hold cubes inside the robot's perimeter while still keeping the robot within the allowed size limits. At the front, we added a V-shaped cardboard piece to push cubes toward the side openings between the robot frame and the cardboard wall. Once the cubes entered this space, the surrounding barrier helped keep them contained. A long strip of cardboard fit through slits in the barrier to hold the wall in place and make the structure more stable.

For the electrical and software design, we used a color sensor to detect the black border of the playing field. When the robot detected black, it would stop driving forward and rotate away from the edge. Since we did not use sensors to detect cubes directly, our strategy was based on covering different areas of the board. To avoid repeating the same path, the code made the robot turn at different angles whenever the black border was detected.

Ideally, the robot would drive around the field, funnel cubes into the side spaces using the V-shaped front, keep them inside the cardboard barrier, and continue changing direction whenever it reached the border. This made our design simple, lightweight, and focused on reliable cube collection.

Design Process Reflection

Throughout the project, our group's design process became focused on keeping the robot as simple and reliable as possible. After talking with the TAs, we realized that a simple design would be easier to test, repeat, and repair during the competition. Since the competition only required cubes to be inside the robot's perimeter, not lifted off the board, we decided to focus on a simple collection system.

At first, we considered a few more complicated intake ideas, including a sweeping arm and a rotating mechanism that could push cubes into our robot's boundary from farther away. However, we eventually decided that simply driving into the cubes and guiding them into our perimeter was the better option. This design had fewer moving parts and gave us less to worry about during testing and competition.

For prototyping, we tested several cardboard shapes, wall sizes, and attachment methods. A big part of this process was figuring out how to keep cubes from falling out when the robot turned. Since we were limited by the robot size requirements, we had to adjust the cardboard walls

carefully. We found that extending the walls slightly past the length of the robot helped keep cubes contained. We also tested different wall heights, especially after deciding to use larger wheels. A taller wall ended up working better because it made the structure more stable and helped the robot hold up better when pushing against cubes or other robots.

When working toward the milestones, we tried to choose the simplest mechanical and software approach that would complete each task. For the movement and sensing milestones, we kept the code focused only on the required actions. For the cube-gathering milestone and final competition, we found that using the color sensor to detect the black border was the most reliable strategy. This allowed the robot to continuously move on the board without needing a more complex cube-detection system.

We did not run into any major roadblocks, but we did change our final design. Originally, we planned to add a net or expandable section at the back of the robot to hold more cubes and reduce clutter near the side walls. After testing, we decided not to use it because it made the robot less stable and did not fit as well with our goal of keeping the robot heavier, sturdier, and more reliable. In the end, our final design was simpler than some of our early ideas, but it was easier to build, test, and fix.

Competition Analysis

Our robot performed well during testing, but we ran into reliability issues during the actual competition. One thing we would change is how we used our budget. We could have used more of it to 3D print parts of the chassis or make a more secure mount for the color sensor. Instead, we relied too much on tape and cardboard to hold parts together, which made the robot less reliable during competition.

Conclusions

If we could do the project over again, we would also reconsider our overall strategy. During competition, we realized that speed was a major factor. A better approach may have been to drive quickly to the center of the field at the start and sweep as many cubes as possible before the other robot could reach them. This strategy could have been simpler because it would rely more on timed movement and distance instead of constantly reacting to the black border with the color sensor.

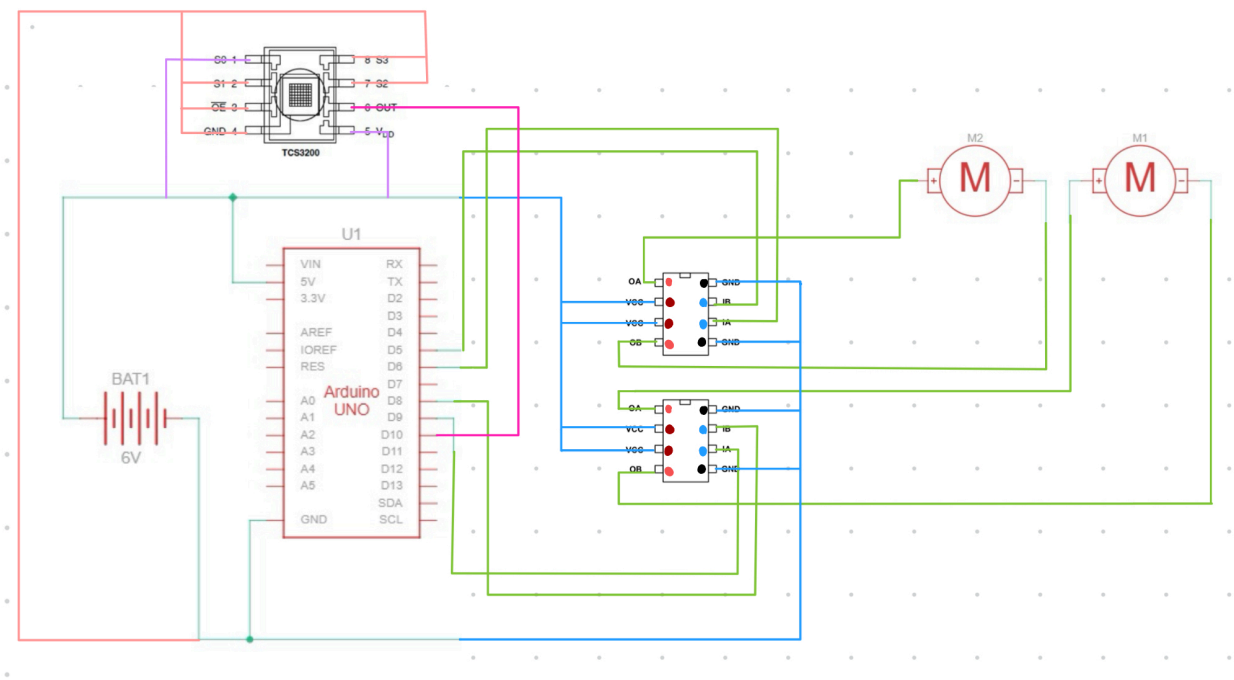
My advice to students doing this project next year would be to optimize the robot to do only what is necessary. Extra turns, extra mechanisms, and extra code can waste time or make the robot less reliable. I would also recommend focusing heavily on making the chassis and sensor mounts sturdy, since small mechanical issues can affect the whole robot. Finally, adding a push button to start the robot would be useful because it would make the start more consistent and help the robot reach the cubes faster at the beginning of each match.

Appendix A: Bill of Materials

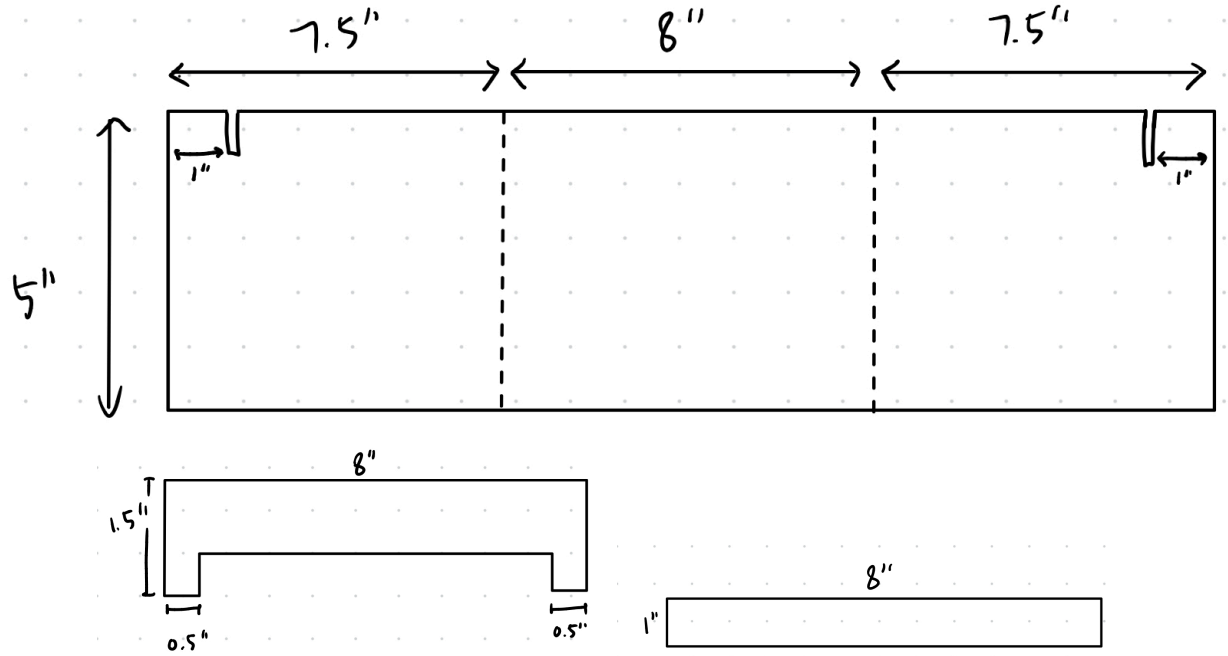
Part No.	Part Name/Description	Vendor	Vendor Part No.	Unit Cost	Quantity	Part(s) Cost
1	Acrylic Sheet (for side panels)	McMaster	8560K358	$6.98	1	$6.98
2	Larger wheels	Polulu	1435	$9.49	1	$9.49
3	cardboard	Lab (but not in lab kit)			1	0
					Total	$16.47

The larger wheels and cardboard were used for the robot, but acrylic was not used as it was too heavy and thick to be utilized well. We changed our design to make it out of cardboard.

Appendix B: Circuit Diagram



Appendix C: CAD files and drawings



Acrylic:

Comparable To	Acrylite, Lucite, Per
Density	0.043 lb/in³
Dielectric Strength	400 V/mil to 500 V/n

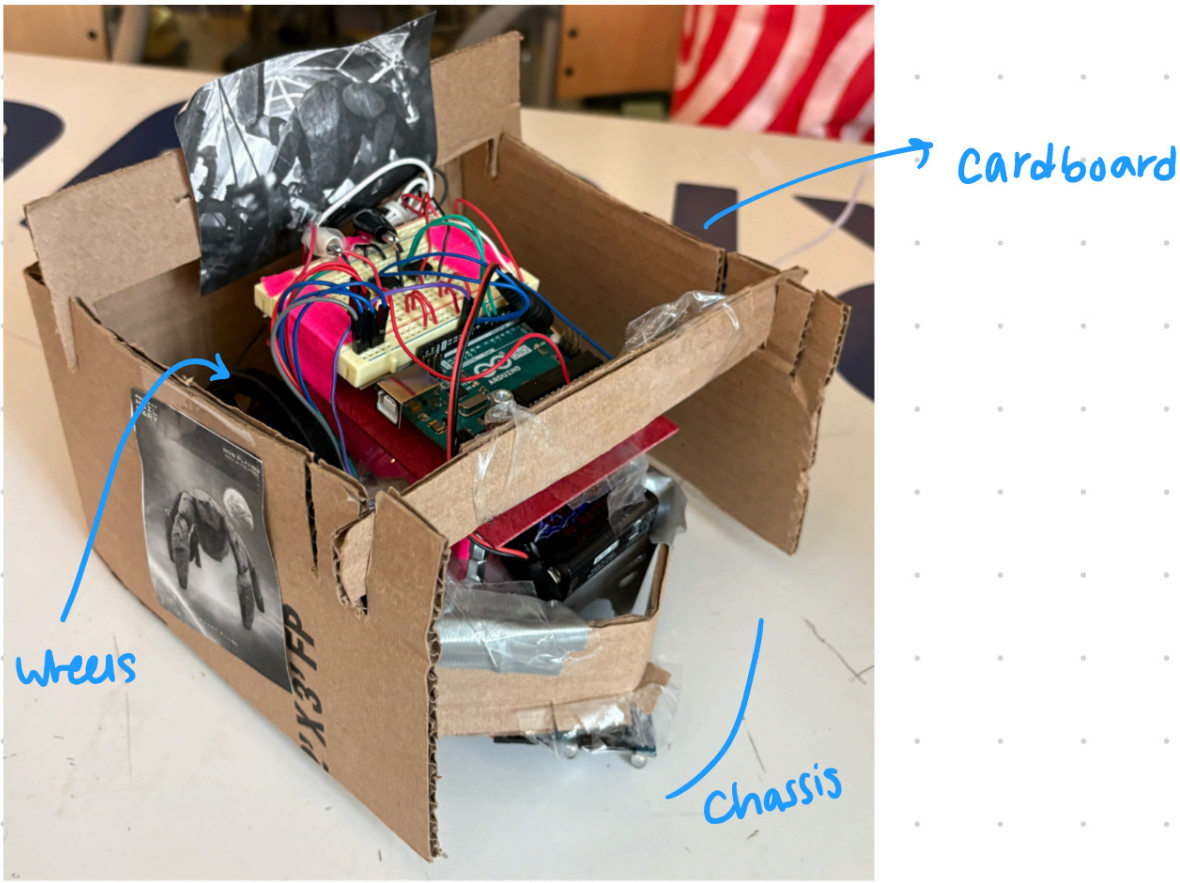
¼”x6”x6”x0.043 =0.387lb = 175.5g

Wheels:

Dimensions

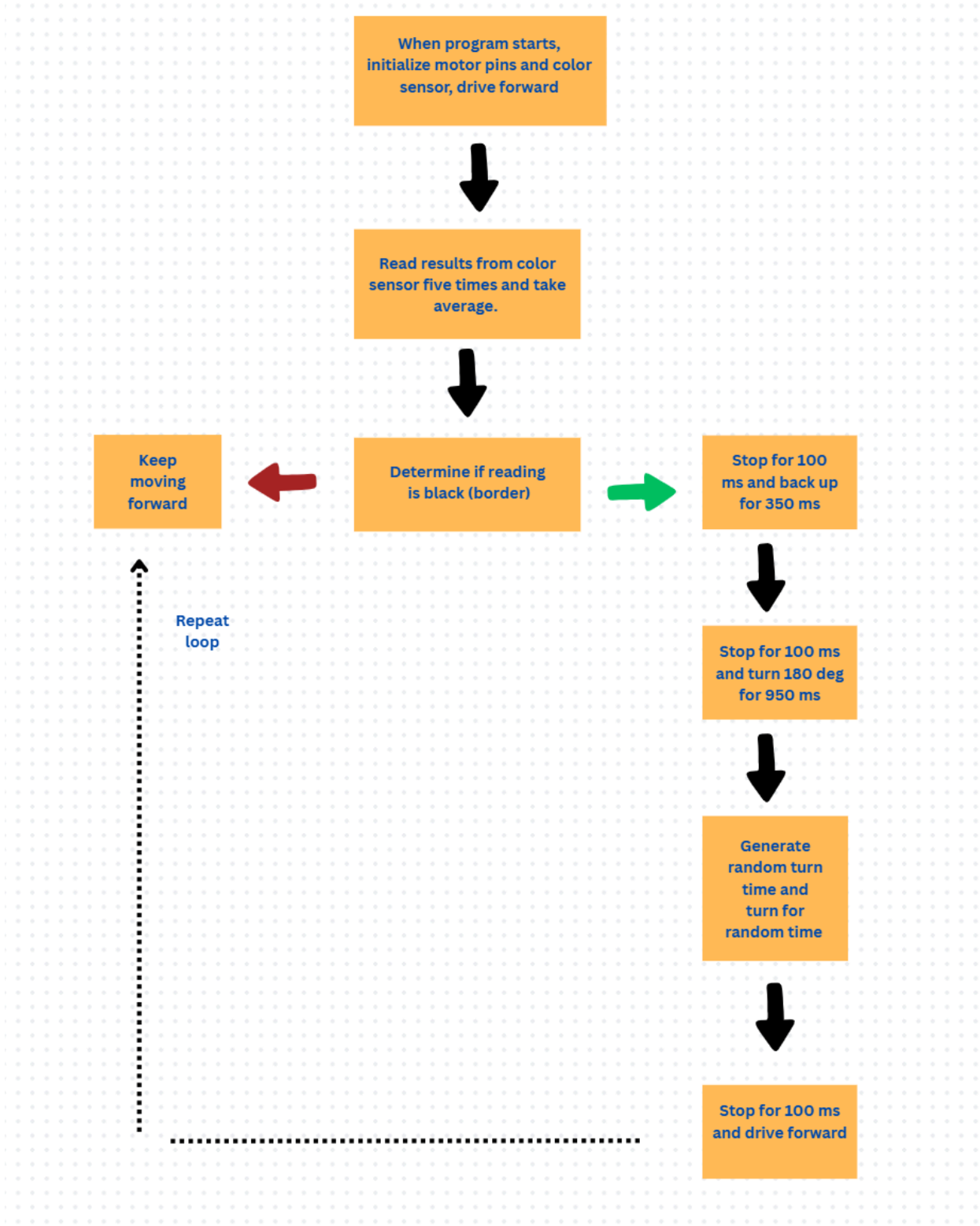
Size:	90 x 10 mm
Weight:	0.8 oz[1]
Shaft diameter:	3 mm[2]

0.8oz = 22.7g



Part No.	Part Name/Description	Weight (g)	Quantity	Total
1	Acrylic Sheet (for side panels)	175.5g	0	0
2	Larger wheels	22.7g	2	45.4h
3	Cardboard (8”x8”x22”)	45.6g	1	45.6
	Chassis total			487.5g

Appendix D: Flowchart



Appendix E: Code

```
#include <Arduino.h>
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

// Variable used to store pulse width from color sensor
volatile unsigned int timer = 0;

// Threshold used to determine if color black is detected
// Color sensor was calibrated by printing sensor readings
// to the serial monitor and comparing the values it measured over
// black and rest of board (yellow/blue).
#define BLACK_THRESHOLD 4000

// Motor control functions
void drive_forward(void);
void drive_backward(void);
void turn_left(void);
void stop_robot(void);

// Color sensor functions
void initColor(void);
unsigned int getColor(void);
unsigned int avgColor(void);
unsigned char isBlack(void);
unsigned int randomTurnTime(void);

// ISR for pin change interrupt on PortB
ISR(PCINT0_vect)
{
  // If sensor output pin is HIGH
  if (PINB & (1 << PINB2)) {
    // Reset Timer1
    TCNT1 = 0;
  } else {
    // Store value of Timer1 when LOW
    timer = TCNT1;
  }
```

```
}

// Initialize color sensor, interrupts, timer
void initColor(void)
{
  // Set D10 = PB2 as input for color sensor output
  DDRB &= ~(1 << DDB2);   // D10 input
  // Set D13 = PB5 as output for LED
  DDRB |= (1 << DDB5);    // D13 LED

  // Pin change interrupts PortB, disable PB2 interrupt initially
  PCICR |= (1 << PCIE0);
  PCMSK0 &= ~(1 << PCINT2);

  // Clear Timer1
  TCCR1A = 0x00;
  TCCR1B = 0x00;
  TCCR1B |= (1 << CS10);

  // Enable global interrupts
  sei();
}

// Measure one pulse from color sensor
unsigned int getColor(void)
{
  timer = 0; // Reset timer value
  TCNT1 = 0; // Reset counter in Timer1

  PCMSK0 |= (1 << PCINT2); // Enable interrupt on D10
  _delay_ms(10); // Delay 0.1 for pulses
  PCMSK0 &= ~(1 << PCINT2); // Disable interrupt after taking meas

  return timer;
}

// Take five readings from sensor and find average
unsigned int avgColor(void)
{
```

```
  unsigned long sum = 0;

  // Repeat fives times
  for (unsigned char i = 0; i < 5; i++) {
    sum += getColor(); // Add sensor reading to current total
    _delay_ms(2); //Delay for readings
  }

  return sum / 5; // Return avg. value
}

// Detect if black
unsigned char isBlack(void)
{
  unsigned int val = avgColor(); // Get averaged reading from sensor

  if (val > BLACK_THRESHOLD) { // Value greater than threshold
    PORTB |= (1 << PORTB5);    // LED on = black
    return 1;                  // Return true
  } else {
    PORTB &= ~(1 << PORTB5);   // LED off = not black
    return 0;                  // Return false
  }
}

int main(void)
{
  // Initialize Arduino
  init();

  // Motor pins
  // D8 = PB0, D9 = PB1
  // D5 = PD5, D6 = PD6
  DDRB |= (1 << DDB0) | (1 << DDB1); // Set as outputs
  DDRD |= (1 << DDD5) | (1 << DDD6); // Set as inputs

  // Initialize color sensor sys
  initColor();
```

```
while (1) // Infinite loop
{
  drive_forward(); // Drive forward

  if (isBlack()) // Check for black
  {
    stop_robot(); // If black, stop robot
    _delay_ms(100); // Delay (tune)

    // Back up from border
    drive_backward(); // Back away from black border
    _delay_ms(350);  // for 350 ms

    stop_robot();  // Stop again
    _delay_ms(100); // For 100 ms

    // Turn around ~180 degrees
    turn_left();
    _delay_ms(950);   // Tune this for exact 180
    stop_robot();     // Stop again
    _delay_ms(100);   // Delay (tune)

    // Go forward for a few seconds
    drive_forward(); // Drive forward
    _delay_ms(300); // For 350 ms

    stop_robot(); // Stop robot
    _delay_ms(100); // For 100 ms

    // Random turn after clearing border
    turn_left();

    // Generate random turn time
    unsigned int t = randomTurnTime(); // Using Timer1
    for (unsigned int i = 0; i < t; i++) { // Turn for random amount of time
      _delay_ms(1);
    }

    stop_robot(); // Stop
```

```
      _delay_ms(100); // For 100 ms
    }

    _delay_ms(10); // Delay for loop
  }
}

// Generate the random turn time
unsigned int randomTurnTime(void)
{
  // If 950 ms ≈ 180 degrees,
  // this gives random turn time from 100 ms to 950 ms.
  // changing it to give random turn max less than 90 degrees =around 45 deg
  unsigned int r = TCNT1; // Using Timer1
  return 100 + (r % 350); // Value between 100 and 450 ms
}

// Drive forward
void drive_forward(void)
{
  // Left motor forward
  PORTB |= (1 << PORTB0);
  PORTB &= ~(1 << PORTB1);

  // Right motor forward
  PORTD |= (1 << PORTD5);
  PORTD &= ~(1 << PORTD6);
}

// Drive backward
void drive_backward(void)
{
  // Left motor backward
  PORTB &= ~(1 << PORTB0);
  PORTB |= (1 << PORTB1);

  // Right motor backward
  PORTD &= ~(1 << PORTD5);
  PORTD |= (1 << PORTD6);
```

```
}

// Turn left
void turn_left(void)
{
  // Left motor backward
  PORTB &= ~(1 << PORTB0);
  PORTB |= (1 << PORTB1);

  // Right motor forward
  PORTD |= (1 << PORTD5);
  PORTD &= ~(1 << PORTD6);
}

// Stop
void stop_robot(void)
{
  // Turn off left motor
  PORTB &= ~((1 << PORTB0) | (1 << PORTB1));
  // Turn off right motor
  PORTD &= ~((1 << PORTD5) | (1 << PORTD6));
}
```